

Задача А. Вместимость площадки

Ограничение по времени: 1 секунда
Ограничение по памяти: 1024 мегабайта

Здравствуйте, уважаемые участники! Мы рады приветствовать вас на нашем соревновании! Пожалуйста, занимайте места в аудиториях, проходите в ближайшую дверь налево. А вы, будьте любезны, в дверь направо. А ваша команда — прямо и налево. Как же много участников сегодня собралось... Минуточку, а мы точно сможем разместить всех желающих?

В распоряжении оргкомитета имеются только два больших компьютерных класса и s учебных аудиторий. Оба компьютерных класса снабжены t широкими столами для троих, а вместимость каждой из учебных аудиторий составляет k человек. Команду из трёх человек можно разместить за одним широким столом в компьютерном классе либо на трёх посадочных местах в одной из учебных аудиторий. При этом разделять команду и рассаживать её участников по разным помещениям запрещается. Также нельзя допускать неполные составы команд.

Организаторы озадачены: сколько же команд они смогут разместить, соблюдая правила? Помогите им выяснить это.

Формат входных данных

Единственная строка входных данных содержит три целых числа s , t , k ($0 \leq s \leq 50$, $5 \leq t \leq 30$, $10 \leq k \leq 50$).

Формат выходных данных

Выведите одно целое число — наибольшее количество команд, которые возможно разместить, не нарушая правил.

Примеры

стандартный ввод	стандартный вывод
20 25 12	130
1 10 10	23
50 30 49	860
16 22 39	252

Замечание

Обратите внимание, что в некоторых случаях не всегда удаётся заполнить все посадочные места в учебных аудиториях. Так, во втором примере останется незанятое место для одного человека.

В третьем примере также останутся незанятые места в каждой учебной аудитории.

Задача В. Хронологический размах

Ограничение по времени: 1 секунда
Ограничение по памяти: 1024 мегабайта

Однажды Никита посетил один знаменитый музей. В музейном зале располагались n статуэток разных культур и эпох. У каждой статуэтки на табличке крупным почерком была указана примерная дата создания. Все статуэтки были выставлены в ряд слева направо от самой древней до самой современной.

Но недавно — какое совпадение, буквально за неделю до сегодняшнего соревнования! — в музее произошло дерзкое ограбление. В репортаже Никита смог разглядеть на крупном плане, что в зале осталось только k статуэток. Но, поскольку злоумышленники спешили, они разбросали то, что не смогли забрать. Поэтому сотрудники музея в спешке расставили оставшиеся статуэтки как попало, без соблюдения хронологического порядка. К сожалению, среди них не оказалось той, что стояла m -й по счёту в изначальной экспозиции — именно она понравилась Никите больше всего. Он не запомнил её год изготовления, но он хочет хотя бы восстановить промежуток возможных дат.

По известным годам изготовления уцелевших k статуэток помогите Никите определить самую раннюю и самую позднюю возможные даты изготовления той самой статуэтки, которая могла бы сейчас находиться на m -й позиции, если бы все n оригинальных экспонатов снова выстроили в том же порядке — от самых древних к самым современным.

Формат входных данных

Первая строка входных данных содержит одно целое число t ($1 \leq t \leq 1000$) — количество наборов входных данных. Далее следуют описания наборов входных данных.

Первая строка очередного набора входных данных содержит три целых числа n , k , m ($2 \leq n \leq 100$, $0 < k < n$, $1 \leq m \leq n$) — изначальное количество статуэток, оставшееся количество статуэток и номер любимой статуэтки Никиты в хронологическом порядке.

Вторая строка содержит k целых чисел $a_1, a_2, \dots, a_k$ ($-10^6 \leq a_i \leq 10^6$) — даты создания уцелевших статуэток, необязательно в хронологическом порядке. Значения a_i могут повторяться.

Гарантируется, что сумма n по всем наборам входных данных не превосходит 10 000.

Формат выходных данных

Для каждого набора входных данных выведите два целых числа — самая ранняя и самая поздняя возможная дата изготовления. Если дата не ограничена снизу, то выведите первым числом -987654321 . Если дата не ограничена сверху, то выведите вторым числом 987654321 .

Примеры

стандартный ввод	стандартный вывод
5	-987654321 1
4 3 1	-6 4
1 3 2	-987654321 10
6 4 3	7 987654321
-3 -6 8 4	-8 987654321
5 2 1	
10 20	
4 3 4	
5 5 7	
6 3 4	
-2 5 -8	
1	-987654321 987654321
100 5 50	
2021 2022 2023 2024 2025	

Задача С. Широкие поля

Ограничение по времени: 1 секунда
Ограничение по памяти: 1024 мегабайта

Рассмотрим уравнение

$$x^n + y^n = z^n,$$

где n — целое число, x, y, z — положительные рациональные числа, представленные в виде дробей:

$$x = \frac{A_x}{B_x}, y = \frac{A_y}{B_y}, z = \frac{A_z}{B_z},$$

и при этом $0 < A_x, B_x, A_y, B_y, A_z, B_z \leq 10^{18}$.

Найдите такой набор целых значений $A_x, B_x, A_y, B_y, A_z, B_z$, при которых левая и правая части уравнения отличаются не более чем на 10^{-50} , либо скажите, что найти такие числа невозможно.

Формат входных данных

В единственной строке записано одно целое число n ($-10 \leq n \leq 10$).

Формат выходных данных

Если ответа нет, выведите слово **No**.

Если же ответ найден, в первой строке выведите слово **Yes**. Во второй строке выведите два целых числа A_x и B_x . В третьей строке выведите два целых числа A_y и B_y , а в четвёртой — два целых числа A_z и B_z .

Можно показать, что если существует хотя бы один ответ, то найдётся и такой, который удовлетворяет ограничениям из условия.

Вы можете выводить каждую букву в любом регистре, строчную или прописную. Например, строки «yEs», «yes», «Yes» и «YES» будут приняты как положительный ответ.

Пример

стандартный ввод	стандартный вывод
1	Yes 20 25 10 26 77 65

Задача D. Поиск пропавшего корабля

Ограничение по времени: 1 секунда
 Ограничение по памяти: 1024 мегабайта

Это интерактивная задача.

Экспедиция научного корабля в океане шла успешно, но в один момент учёные попали в зону, в которой потеряли связь с берегом. Половина беды в том, что передать на материк полученные сведения об исследованиях не получится. Увы, придётся возвращаться. Но вторая половина беды в том, что из-за потери связи нет надёжного способа вернуться обратно! Группа учёных, которая осталась на берегу, уже выслала специальный дрон, который найдёт местоположение корабля, а после проведёт к берегу.

Исследуемая зона океана представлена на морской карте таблицей $n \times n$ клеток. Корабль занимает последовательные $k > 1$ клеток вертикально или горизонтально. Строки таблицы нумеруются сверху вниз от 1 до n , столбцы нумеруются слева направо от 1 до n . За один запрос дрон может выбрать ячейку (r, c) . Если корабль имеет хотя бы одну клетку в r -й строке **или** в c -м столбце, то на этом поиски успешно заканчиваются. Иначе корабль после запроса может либо остаться на месте, либо сдвинуться на одну клетку в одну из двух сторон в зависимости от его ориентации. Корабль не может покинуть исследуемый район океана, все клетки корабля должны оставаться внутри таблицы.

Пример для $n = 5$, $k = 2$ и проверки дроном позиции $(r, c) = (1, 2)$ показан на рисунке 1. Стрелками указано, куда может переместиться корабль после запроса.

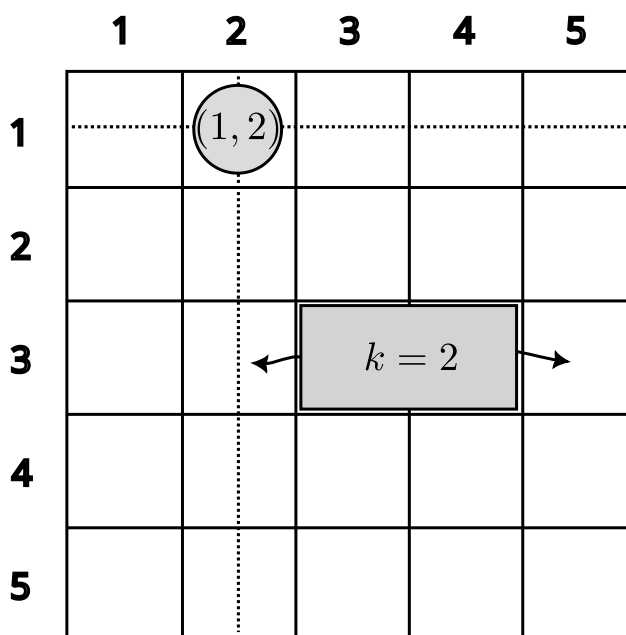


Рисунок 1. Пример хода

Необходимо за минимальное возможное количество ходов найти корабль — попасть в строку или столбец, в котором точно есть хотя бы одна клетка корабля.

Формат входных данных

Первая строка входных данных содержит два целых числа n и k ($3 \leq n \leq 10$, $2 \leq k \leq n$), задающих размер таблицы и длину корабля.

Протокол взаимодействия

Для того чтобы сделать запрос, выведите два целых числа r и c ($1 \leq r, c \leq n$). Ответ на запрос будет записан в стандартный поток ввода: вы получите число 0, если корабль ещё не обнаружен, либо число 1, если обнаружен. Как только вы прочитали число 1, программа должна быть немедленно завершена. Если же вы прочитали число 0, вы можете сделать новый запрос.

В случае, если вы сделаете больше запросов, чем минимальное необходимое количество для гарантированного нахождения корабля, или координаты запроса будут вне исследуемой зоны океана, ваша программа получит вердикт «Неправильный ответ» (Wrong Answer), а ответом на запрос будет число -1 . При прочтении ответа -1 программа должна быть немедленно завершена. Если ваше решение не закончит выполнение сразу после нахождения корабля, либо закончит раньше нахождения корабля, либо продолжит выполнение после ответа -1 , то оно будет засчитано как неверное, а вердикт может быть неопределённым.

После каждого запроса не забудьте вывести перевод строки и выгрузить содержимое буфера в поток вывода. Для этого вы можете воспользоваться следующими функциями:

- `fflush(stdout)` или `std::cout.flush()` в C++;
- `System.out.flush()` в Java;
- `flush(output)` в Pascal;
- `stdout.flush()` в Python;
- смотрите документацию для других языков.

Пример

стандартный ввод	стандартный вывод
3 2 1	2 2

Замечание

В примере мы делаем один запрос в клетку $(2, 2)$, которая позволяет за один ход сразу найти корабль независимо от его местоположения. В ответ на этот ход прочитано число 1, после чего программа завершает своё выполнение, потому что обнаружила корабль.

Задача Е. Салат «Цезарь»

Ограничение по времени: 2 секунды
Ограничение по памяти: 1024 мегабайта

— Как решал?
— Брут.
— И я брут!
— И ты брут?..

Из бесед участников после соревнования

Представляем вашему вниманию инновационный способ шифрования, вдохновлённый балансом античной мудрости и современной простоты!

Вы, конечно же, знакомы с шифром Цезаря. Его суть исключительно проста: каждая буква в сообщении сдвигается на одно и то же значение вперёд по алфавиту. Если при сдвиге мы вышли за пределы алфавита, то тогда отсчёт продолжается с его начала. Так, слово `fusion` при сдвиге, равном 6, превращается в слово `layout`. А чтобы зашифрованное послание расшифровать, нужно сдвинуть каждую букву на то же значение, но уже не вперёд, а назад по алфавиту.

Новый метод шифрования называется *шифр салата «Цезарь»*. Он очень похож на свой древний прототип, только надёжнее! Его применяют к словам, состоящим из букв от `a` до `z`. Каждой букве латинского алфавита соответствует числовой код от 0 до 25: так, буква `a` имеет код 0, буква `b` — код 1, `c` — 2 и так далее до буквы `z`, имеющей код 25. Работает шифр салата «Цезарь» так:

1. Пусть $(a_1, a_2, \dots, a_k)$ — числовые коды букв исходного сообщения длины k . Например, слову `delta` соответствует последовательность кодов $(3, 4, 11, 19, 0)$.
2. Выберем сдвиг x — целое число от 1 до 25 включительно. Заметьте, что, как и в оригинальном шифре Цезаря, нулевой сдвиг выбирать нельзя из соображений безопасности.
3. Присвоим $b_1 := (a_1 + x) \bmod 26$. Здесь $p \bmod m$ означает остаток от деления p на m .
4. Далее для каждого i от 2 до k присвоим $b_i := (a_i + b_{i-1} + x) \bmod 26$.
5. Последовательность кодов $(b_1, b_2, \dots, b_k)$ соответствует зашифрованному сообщению из k букв. Так, после шифрования слова `delta` со сдвигом $x = 20$ получится $(23, 21, 0, 13, 7)$, что преобразуется в слово `xvanh`.

А теперь с места в карьер — самое время применить новые знания на практике! Вам предстоит разработать протокол передачи цифрового ключа от электронной копилки «Салатница». Цифровой ключ состоит из n слов, каждое из которых содержит k латинских букв. Среди этих слов могут быть повторяющиеся, но их порядок в общем и целом значения не имеет. В чистом виде цифровой ключ не передают по открытым каналам — перед этим к нему примешивают n новых слов длины k , которые являются исходными словами, пропущенными через шифр салата «Цезарь». При этом каждое из n слов разрешается шифровать с разными сдвигами. Полученный набор из $2n$ слов, перемешанный в произвольном порядке, называется цифровым ключом с *примесью*.

Ваша задача — реализовать обе части протокола. В первой части по заданному цифровому ключу необходимо сгенерировать ключ с примесью, а во второй — выделить из ключа с примесью, сгенерированного вашим алгоритмом, все слова исходного цифрового ключа. Детали реализации оставляем на ваше усмотрение.

Формат входных данных

Программа будет запущена на каждом тесте дважды. На первом запуске программе на вход будут поданы исходные данные, а на втором — данные, полученные после ваших преобразований.

Каждый тест состоит из нескольких наборов входных данных. В первой строке находятся одно слово S из прописных латинских букв и одно целое число t ($1 \leq t \leq 1000$) — количество наборов

входных данных. Если $S = \text{ENCODE}$, то это **первый запуск** и для каждого набора входных данных требуется сгенерировать цифровой ключ с примесью. Если $S = \text{DECODE}$, то это **второй запуск** и для каждого набора входных данных из ключа с примесью требуется выделить слова исходного цифрового ключа. Далее следует описание t наборов входных данных.

Первая строка каждого набора входных данных содержит два целых числа n и k ($1 \leq n \leq 12$, $1 \leq k \leq 20$) — количество слов в исходном цифровом ключе и длина каждого из этих слов.

Если $S = \text{ENCODE}$, каждая из следующих n строк содержит по одному слову из k строчных латинских букв. Эти n слов задают исходный цифровой ключ. Некоторые слова могут совпадать.

Если же $S = \text{DECODE}$, каждая из следующих $2n$ строк содержит по одному слову из k строчных латинских букв. Эти $2n$ слов задают цифровой ключ с примесью, выведенной вашей программой на первом запуске. Порядок слов в ключе с примесью может быть произвольным.

Формат выходных данных

Если $S = \text{ENCODE}$, для каждого набора входных данных выведите n слов длины k — слова, зашифрованные шифром салата «Цезарь». Разрешается выбирать разные сдвиги для каждого из слов, но порядок вывода зашифрованных слов должен соответствовать порядку исходных слов. Эти зашифрованные слова будут примешаны к n исходным словам перед вторым запуском.

Если же $S = \text{DECODE}$, для каждого набора входных данных выведите n слов длины k — восстановленный цифровой ключ, который должен совпасть с исходным ключом. Вы можете выводить слова цифрового ключа на втором запуске в любом порядке.

Регистр букв имеет значение: выводимые латинские буквы должны быть строчными.

Пример

стандартный ввод	стандартный вывод
ENCODE 2 4 5 delta alpha alpha prime 1 20 belarusqualification	xvanh zjxdc kfevf rkuio hriolljfflcbpxdcqkd
DECODE 2 4 5 alpha prime kfevf delta zjxdc alpha rkuio xvanh 1 20 hriolljfflcbpxdcqkd belarusqualification	alpha delta prime alpha belarusqualification

Замечание

Пример иллюстрирует формат входных и выходных данных для обоих запусков. Описания данных для первого и второго запусков разделены пустой строкой для наглядности.

Для первого набора входных данных для слов выбраны сдвиги 20, 25, 10 и 2 соответственно.

Для второго набора входных данных после первого запуска слово зашифровано со сдвигом 6.

Задача F. Графический ключ

Ограничение по времени: 1 секунда
Ограничение по памяти: 1024 мегабайта

Не так давно председатель жюри одной престижной олимпиады — человек видный и уважаемый, самый уважаемый и самый видный среди членов жюри! — купил себе новый телефон. Ковыряясь в настройках телефона, председатель обнаружил штуку, которая называется графический ключ. Он стал разбираться самостоятельно и, в конце концов, изобрёл такую оригинальную трактовку графического ключа.

Представим себе таблицу 3×3 , состоящую из трёх строк одинаковой высоты и трёх столбцов одинаковой ширины. На пересечении каждой строки и каждого столбца ровно в центре ячейки имеется точка. Стало быть, всего таких точек в таблице девять. *Графический ключ* — это ломаная, соединяющая одну или несколько таких точек в таблице. Графический ключ строится так:

1. На экране помечается одна из точек в таблице в качестве начальной точки ломаной. На эту точку ставится палец председателя.
2. Далее председатель принимает взвешенное решение: либо оторвать палец от экрана и обратно его больше не ставить, либо провести его под углом, кратным 45° , строго вдоль одного из восьми направлений: вверх, вниз, вправо или влево, а также наискосок вверх-вправо, вниз-вправо, вниз-влево или вверх-влево.
3. Как только при движении пальца на пути встретилась ранее не помеченная точка в таблице, она помечается, добавляется в ломаную, а палец останавливается на ней — переходим к шагу 2. Если же на пути точек не было, то палец выползает за пределы экрана.
4. Построение графического ключа заканчивается, когда палец отрывается от экрана или выползает за его пределы при движении в каком-нибудь направлении. Помеченные к этому моменту точки с учётом порядка их пометки и являются графическим ключом.

Вероятно, ранее вы сталкивались с графическими ключами, которые строятся по слегка иным принципам. Но ничего не попишешь, у нашего председателя свои принципы и свои возможности. Никакими другими траекториями (в том числе и «буквой Г», то есть на две клетки вперёд и одну в сторону), кроме тех, что описаны выше, ходить по экрану между двумя точками он не умеет — пальцы уже не те, всё время скользят, задевают другие точки, а то и вовсе с экрана выползают. Но зато умудрённый жизненным опытом председатель подметил такой факт, что ломаная может пересекать одну и ту же точку в таблице несколько раз. Конечно же, считается только первое её пересечение с ломаной, а все остальные игнорируются.

Узнайте, сколько различных ключей с n помеченными точками в таблице 3×3 может построить председатель, если будет действовать по своей схеме.

Формат входных данных

В единственной строке входных данных записано одно целое число n ($1 \leq n \leq 9$) — количество помеченных точек в ключе.

Формат выходных данных

Выведите одно целое число — искомое количество различных графических ключей.

Примеры

стандартный ввод	стандартный вывод
2	40
3	176

Задача G. Кухня

Ограничение по времени: 2 секунды
Ограничение по памяти: 1024 мегабайта

Зайдя к себе на кухню, Петя обнаружил ужасный сюрприз: там завелись тараканы!

Не медля ни минуты, он разместил ядовитую приманку для их истребления в нескольких точках кухни. Так как сделал он это в спешке, есть шанс, что какие-то тараканы выживут. Помогите ему предсказать исход событий.

По прямоугольной форме тела Петя смог определить, что тараканы принадлежат к редкому виду манхэттенских тараканов. Если такой таракан бежит из точки (x_1, y_1) в точку (x_2, y_2) , то сперва он бежит по прямой в точку (x_2, y_1) , а уже оттуда — по прямой в (x_2, y_2) .

Каждый таракан выбирает такую приманку, время его движения до которой как можно меньше (а если таких приманок несколько, то он выбирает ту, у которой координаты (x_j, y_j) лексикографически минимальны), и движется к ней со скоростью 1 метр в секунду.

Когда таракан добегают до приманки, он мгновенно её съедает, после чего все бежавшие к ней тараканы меняют свою цель по описанному выше алгоритму. Если несколько тараканов добегают до одной приманки одновременно, то в пылу внутривидовой борьбы её съедает тот, чьи изначальные координаты были даны во входных данных раньше.

Поскольку эти тараканы необычайно живучи, они могут выжить, если съедят менее a кусочков яда, и даже после съедения данного количества они ещё активны на протяжении t секунд до своей смерти. Если таракан добегают до приманки в момент смерти, то он ещё успевает её съесть.

Формат входных данных

Первая строка содержит четыре целых числа n, m, a, t ($1 \leq n \leq 100, 1 \leq m \leq 1000, 1 \leq a \leq 1000, 0 \leq t \leq 10^9$), что означает, что всего на кухне n тараканов, m приманок, каждый таракан может съесть a приманок и потом ещё прожить t секунд.

Вторая строка содержит два целых числа W и H ($1 \leq W, H \leq 10^9$) — размеры прямоугольной кухни Пети в метрах.

Каждая из последующих n строк содержит два целых числа x, y ($0 \leq x \leq W, 0 \leq y \leq H$) — изначальные координаты очередного таракана.

Каждая из последующих m строк содержит два целых числа x, y ($0 \leq x \leq W, 0 \leq y \leq H$) — координаты очередной приманки. Гарантируется, что никакие две приманки не находятся в одной точке.

Формат выходных данных

Для каждого таракана выведите одно число — количество секунд, через которое он погибнет от яда, либо -1 , если таракан выживет. Ответы для тараканов должны быть даны в том же порядке, в котором координаты тараканов следуют во входных данных.

Пример

стандартный ввод	стандартный вывод
3 3 1 2	-1
4 4	9
0 0	4
0 1	
1 1	
2 2	
3 3	
4 4	

Замечание

Координаты (x_1, y_1) лексикографически меньше координат (x_2, y_2) , если либо $x_1 < x_2$, либо $x_1 = x_2$ и $y_1 < y_2$.

Задача Н. Реактор

Ограничение по времени: 1 секунда
Ограничение по памяти: 1024 мегабайта

Проектирование надёжного и эффективного реактора состоит из множества этапов, и вам предстоит помочь с планированием одного из них.

Имеется каркас, представляющий собой сетку размера $n \times m$. Некоторые из этих клеток содержат несущие стены для надёжности конструкции и не могут быть использованы для других нужд. Остальные клетки необходимо заполнить топливными стержнями и охлаждающими элементами.

В соответствии с требованиями техники безопасности каждый топливный стержень должен соседствовать как минимум с одним охлаждающим элементом. Более формально, их клетки должны иметь общую сторону.

Чем больше в конструкции топливных стержней, тем больше получится извлечь энергии из реактора, поэтому ваша задача — найти любую из конфигураций с максимальным количеством топливных стержней при соблюдении всех условий.

Формат входных данных

Первая строка содержит два целых числа n и m ($1 \leq n \leq 10$, $1 \leq m \leq 50$) — размеры каркаса реактора.

Следующие n строк содержат по m символов, описывающих клетки реактора. Символ '.' соответствует свободной клетке, а символ '#' — несущей стеной.

Формат выходных данных

Выведите nm символов, по m символов в каждой из n строк — оптимальную расстановку элементов реактора. Символы могут принимать следующие значения:

- '#', если в клетке находится несущая стена;
- '0', если вы разместили в клетке топливный стержень;
- '1', если вы разместили в клетке охлаждающий элемент;
- '.', если клетка осталась пустой.

Гарантируется, что хотя бы одно корректное решение существует. Если оптимальных решений несколько, выведите любое из них.

Пример

стандартный ввод	стандартный вывод
6 9	001000100
.....	100010001
.....	0010#0100
...#....	0010#0100
...#....	100010001
.....	001000100
.....	

Задача I. Справочная

Ограничение по времени: 1.5 секунд
Ограничение по памяти: 1024 мегабайта

Работа в городском справочном бюро — дело непростое! Вам, как его сотруднику, постоянно приходят вопросы, на которые едва ли можно мгновенно дать ответ.

Вася тоже не любит отвечать на вопросы. Зато он любит, когда это делают компьютерные программы. Вот и сейчас у него есть массив a из n целых чисел $a_1, a_2, \dots, a_n$, и он хочет написать программу, которая умеет отвечать на следующие запросы:

- 1 i $value$ — присвоить элементу a_i значение $value$;
- 2 l r — посчитать *сумму Васи* для чисел $a_l, a_{l+1}, \dots, a_r$.

Вася не считает все числа одинаково хорошими. Ему больше нравятся числа, стоящие справа от других чисел — причём ровно в два раза! Поэтому он определяет *сумму Васи* для чисел $x_1, x_2, \dots, x_k$ как величину

$$\sum_{i=1}^k 2^{i-1} \cdot x_i = x_1 + 2x_2 + 4x_3 + \dots + 2^{k-1}x_k.$$

Даже если Вася не смог написать такую программу, не беда — где, как не в *справочном бюро*, могут помочь посчитать что-то в пользу чисел справа? Теперь бремя решения перешло именно вам! Чтобы облегчить подсчёты, вам нужно для каждой *суммы Васи* лишь сказать, положительная она, отрицательная или равна нулю.

Формат входных данных

Первая строка входных данных содержит одно целое число t ($1 \leq t \leq 3 \cdot 10^5$) — количество тестовых случаев. Далее следуют t описаний тестовых случаев в формате, описанном ниже.

Первая строка тестового случая содержит два целых числа n и q ($1 \leq n, q \leq 3 \cdot 10^5$) — количество чисел в массиве Васи и количество запросов, соответственно.

В следующей строке записано n целых чисел $a_1, a_2, \dots, a_n$ ($-10^9 \leq a_i \leq 10^9$) — элементы массива Васи в порядке слева направо.

Каждая из следующих q строк описывает очередной запрос либо в формате 1 i $value$, либо в формате 2 L R ($1 \leq i \leq n$, $-10^9 \leq value \leq 10^9$, $1 \leq L \leq R \leq n$). Запрос первого типа означает, что нужно поменять значение элемента a_i на $value$. Запрос второго типа означает, что нужно найти знак *суммы Васи* для чисел $a_L, a_{L+1}, \dots, a_R$ (конечно, именно в этом порядке).

Гарантируется, что суммы каждой из величин n и q по всем тестовым случаям не превосходят $3 \cdot 10^5$, и что в каждом тестовом случае есть хотя бы один запрос второго типа.

Формат выходных данных

В каждом тестовом случае выведите по одному числу для каждого запроса типа 2. Если сумма Васи в данном запросе положительна, выведите 1; если отрицательна, выведите -1; в противном случае выведите 0.

Пример

стандартный ввод	стандартный вывод
2	1
4 7	0
9 -4 2 -1	-1
2 1 4	0
2 2 3	-1
2 2 4	-1
1 1 8	
2 1 4	
1 2 -5	
2 1 4	
7 1	
2025 10 26 -15 59 59 -76	
2 1 7	

Замечание

В первом тестовом случае *суммы Васи* равны соответственно 1, 0, -4, 0, -2.

Во втором тестовом случае *сумма Васи* равна -3.

Обратите внимание на то, что в этой задаче входные и выходные данные могут иметь большой размер. Рекомендуем использовать методы ускорения чтения и записи данных, которые имеются в используемом вами языке программирования.

Задача J. Искусство производства

Ограничение по времени: 2 секунды
Ограничение по памяти: 1024 мегабайта

Пусть x — целое число, которое записывается n одинаковыми цифрами a в системе счисления по основанию r . Также пусть y — целое число, запись которого состоит из m цифр b в r -ичной системе счисления.

Найдите сумму цифр записанного в r -ичной системе счисления числа $z = x \cdot y$, а кроме того, выведите эту сумму цифр тоже в r -ичной системе счисления.

Формат входных данных

В первой строке находится одно целое число t ($1 \leq t \leq 10\,000$) — количество наборов входных данных. Далее следует описание t наборов входных данных.

Единственная строка каждого набора входных данных содержит пять целых чисел n, m, a, b, r ($n, m \geq 1, n + m \leq 10^{18}, 1 \leq a, b < r \leq 10$).

Формат выходных данных

Для каждого набора входных данных выведите в отдельной строке одно целое число — ответ на задачу в системе счисления по основанию r .

Пример

стандартный ввод	стандартный вывод
5	27
2 3 6 6 10	84
6 11 7 7 10	204
20 25 2 6 10	100101
13 37 1 1 2	71
57 1 4 2 8	

Замечание

В первом наборе входных данных имеем $x = 66_{10}, y = 666_{10}, z = 43\,956_{10}$. Сумма цифр z равна 27.

В четвёртом наборе входных данных сумма цифр z в двоичной записи равна $37_{10} = 100101_2$.

Задача К. Реконструкция крепостной стены

Ограничение по времени: 2 секунды
Ограничение по памяти: 1024 мегабайта

В древности люди возводили вокруг своих городов крепостные стены для защиты от нападений извне. Когда к городу подбирался супостат, городские ворота закрывались, и неприятелю ничего не оставалось, кроме как лезть на стену. А наверху его уже поджидали разъярённые защитники города, встреча с которыми не сулила ничего хорошего.

Со временем города разрастались настолько, что внутри крепостных стен становилось тесно. Поэтому стены понемногу начали разбирать. В наши дни в мире осталось не так уж много городов, сохранивших хотя бы часть бывших крепостных стен. Конечно, защитной функции они уже не несут, зато имеют историческую ценность и обеспечивают приток туристов в страну и налогов в казну.

В Министерстве архитектуры и строительства предложили реконструировать древнюю крепостную стену в одном из культурных центров страны. С течением веков стена изрядно пострадала, а кое-где и вовсе разрушилась. Перед реставрацией важно оценить стоимость работ.

Крепостную стену можно представить в виде n подряд идущих вертикальных столбов, для удобства пронумерованных от 1 до n . Столб с номером i имеет высоту a_i . Городские архитекторы предложили q планов реставрации стены. План с номером j заключается в том, чтобы сделать высоту всех подряд идущих столбов, пронумерованных от ℓ_j до r_j включительно, одинаковой и равной h_j . Если высота столба меньше h_j , то предлагается нарастить его с помощью современных строительных материалов, а если больше h_j , то придётся разобрать часть кладки.

Изменение высоты любого столба на 1, как в сторону увеличения, так и в сторону уменьшения, стоит одну безусловную единицу. Подсчитайте для каждого из предложенных планов, сколько безусловных единиц стоит переделать фрагмент стены. Учитывайте, что это только планы и никаких реальных изменений со стеной при этом не происходит.

Формат входных данных

Первая строка входных данных содержит два целых числа n и q ($1 \leq n, q \leq 300\,000$) — количество столбов в стене и количество планов.

Вторая строка содержит n целых чисел $a_1, a_2, \dots, a_n$ ($1 \leq a_i \leq 10^9$) — высоты столбов.

Далее следуют q строк, j -я из которых содержит три целых числа ℓ_j, r_j, h_j ($1 \leq \ell_j \leq r_j \leq n$, $1 \leq h_j \leq 10^9$) — описание плана с номером j .

Формат выходных данных

Выведите q строк, j -я из которых содержит одно целое число — стоимость реконструкции стены в соответствии с j -м планом.

Примеры

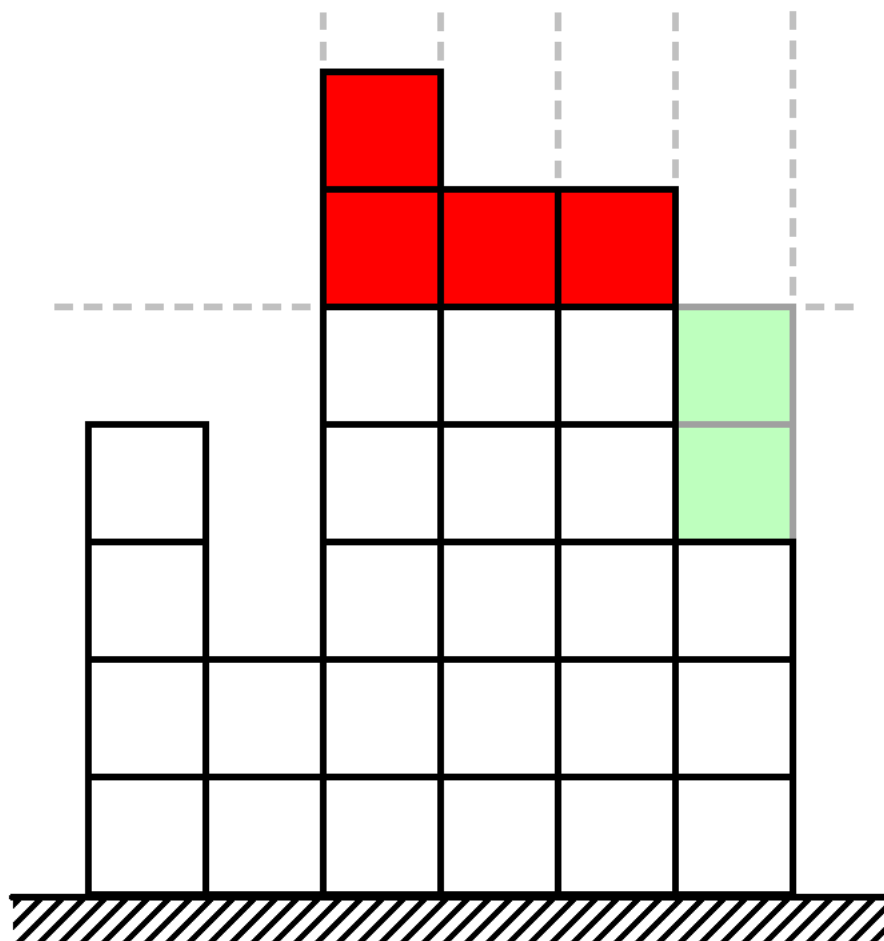
стандартный ввод	стандартный вывод
4 3 3 1 1 2 1 4 1 1 4 2 1 4 3	3 3 5
6 5 4 2 7 6 6 3 2 4 7 3 6 5 1 5 1 2 6 8 1 4 3	6 6 20 16 9

Замечание

Обратите внимание на то, что в этой задаче входные и выходные данные могут иметь большой

размер. Рекомендуем использовать методы ускорения чтения и записи данных, которые имеются в используемом вами языке программирования.

На рисунке ниже приведена графическая иллюстрация второго примера с планом под номером 2: высоту 3-го столбца нужно уменьшить на 2, 4-го и 5-го столбцов — на 1, а высоту 6-го столбца нужно увеличить на 2, чтобы высоты всех этих столбцов стали равны 5.



Задача L. Ожидается минимум

Ограничение по времени: 3 секунды
Ограничение по памяти: 1024 мегабайта

Даны целые числа n , x , y , причём $x \neq y$. Определим функцию $f(p)$, где p может принимать действительные значения между 0 и 1, следующим образом.

Сгенерируем n чисел $a_1, a_2, \dots, a_n$ независимо друг от друга следующим образом: положим значение равным x с вероятностью p или равным y с вероятностью $1 - p$. После этого рассмотрим следующую величину:

$$c_{\min}(p) = \min_{i=1}^{n-1} |(a_1 + a_2 + \dots + a_i) - (a_{i+1} + a_{i+2} + \dots + a_n)|$$

Другими словами, значение $c_{\min}(p)$ равно минимально возможному модулю разности сумм на префиксе и суффиксе последовательности $a_1, a_2, \dots, a_n$ при её оптимальном разбиении на **непустые** префикс и суффикс. Определим $f(p)$ как математическое ожидание величины $c_{\min}(p)$.

Ваша задача — найти $f(p)$ для нескольких **рациональных** значений p .

Формат входных данных

Первая строка содержит одно целое число t ($1 \leq t \leq 2500$) — количество тестовых случаев, описания которых следуют далее.

Первая строка описания содержит четыре целых числа n , x , y , q ($2 \leq n \leq 5000$, $1 \leq x, y \leq 10^6$, $x \neq y$, $1 \leq q \leq 5000$) — количество генерируемых чисел, два возможных значения для них и количество запросов соответственно.

Каждая из следующих q строк содержит два целых числа a и b ($0 \leq a \leq 10^6$, $\max\{1, a\} \leq b \leq 10^6$). Это означает, что в очередном запросе вам нужно найти $f(a/b)$.

Гарантируется, что суммы каждой из величин n и q по всем тестовым случаям не превосходят 5000.

Формат выходных данных

Для каждого тестового случая выведите q чисел — значения $f(p)$ для запрашиваемых $p = a/b$ по модулю 998 244 353.

Формально, пусть $M = 998\,244\,353$. Можно показать, что ответ может быть представлен в виде несократимой дроби P/Q , где P и Q — целые числа и $Q \not\equiv 0 \pmod{M}$. Вместо значения $f = P/Q$ нужно вывести такое целое число r , что $0 \leq r < M$ и $r \cdot Q \equiv P \pmod{M}$.

Пример

стандартный ввод	стандартный вывод
2	1
2 2 4 3	0
1 2	0
1 1	748683269
0 1	838032300
4 5 12 3	857866244
2 4	
4 6	
3 4	

Замечание

В первом запросе первого тестового случая с вероятностью $\frac{1}{4}$ возможна каждая из следующих последовательностей: (2, 2); (2, 4); (4, 2); (4, 4). Для каждой из них возможно только одно разбиение на непустые префикс и суффикс: на префикс и суффикс длины 1. Для первой и последней последовательности $c_{\min}$ равно 0; для второй и третьей $c_{\min}$ равно 2. Поэтому ответ равен $\frac{0+2+2+0}{4} = 1$.